



UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3069 – Computación Paralela y Distribuida

Proyecto 1 **Screensaver "Galaxia" paralelizado con OpenMP**

Informe de proyecto

Sebastian Estrada – 21405

Maria Jose Yee – 231193

Guatemala, septiembre de 2026

Índice

1. Introducción

2. Antecedentes: marco conceptual

3. Diseño e implementación

3.1 Cómo funciona la galaxia

3.2 Estructura del código

3.3 PCAM aplicado

3.4 Versiones paralelas

3.5 Cómo comprobamos que el resultado no cambia

4. Metodología de medición

5. Resultados y discusión

5.1 Resultados

5.2 Discusión

6. Conclusiones

7. Referencias

Anexo 1 – Diagrama de flujo del programa

Anexo 2 – Catálogo de funciones

A2.1 Funciones compartidas ('saver_common.hpp')

A2.2 Funciones propias de cada versión

A2.3 Estructuras de datos principales

Anexo 3 – Bitácora de pruebas

A3.1 Mediciones individuales (10 por prueba)

A3.2 Resumen por prueba

A3.3 Capturas de las mediciones

A3.4 Búsqueda del N máximo a 60 FPS

A3.5 Experimento de política de espera

1. Introducción

El proyecto trata de hacer un *screensaver* que corriera en paralelo, empezamos ideando unos bloques que rebotaban por la pantalla y se iban comiendo entre ellos. Conforme fuimos avanzando y al mezclar ideas descubrimos que una galaxia se veía mejor y además involucra muchos más cálculos, que era lo que buscábamos para aprovechar al máximo el paralelismo. La versión final es un disco de polvo de colores que gira. Dentro de él se forman estrellas que atraen el polvo con su gravedad, todo esto lo programamos en C++, el programa muestra los FPS, valor de N y los hilos utilizados estos valores se controlan mediante los parámetros al ejecutar el programa.

Iniciamos por la versión secuencial, que fue nuestra referencia para todas las mediciones, a partir de esta versión utilizamos PCAM para decidir qué partes de cada cuadro se podían repartir entre hilos. Después paralelizamos con OpenMP en dos rondas. Primero hicimos una v1 bastante directa y luego una v2 en la que corregimos lo que en la v1 estaba generando problemas. Para comparar las tres usamos el *speedup* y la eficiencia con 10 corridas por cada versión y sus configuraciones respectivas.

2. Antecedentes: marco conceptual

OpenMP. Es una API pensada para escribir programas paralelos de memoria compartida en C, C++ y Fortran (OpenMP Architecture Review Board, 2021). Al entrar en una región 'parallel' el programa se divide en un equipo de hilos. Los hilos se reparten el trabajo con construcciones como 'for' o 'single' y al terminar cada construcción se esperan unos a otros en una barrera antes de seguir (*join*). Lo que más hay que cuidar son las condiciones de carrera. Aparecen cuando dos hilos acceden al mismo dato y al menos uno lo cambia sin que nada los coordine (Pacheco y Malensek, 2022).

PCAM. (Foster, 1995) divide el diseño de un algoritmo paralelo en cuatro etapas. Primero se parte el problema en tareas pequeñas. Luego se revisa qué información tiene que pasar de unas tareas a otras. Después se juntan las tareas cuando coordinarlas por separado saldría más caro (Hablando de complejidad algorítmica) que el trabajo mismo. Al final se decide qué procesador hace qué.

Speedup, eficiencia y ley de Amdahl. El speedup compara el tiempo secuencial con el paralelo ($S = T_s / T_p$) y la eficiencia lo divide entre el número de hilos ($E = S / p$) (Pacheco y Malensek, 2022). Amdahl (1967) definió un límite para el speedup. Si una

fracción f del programa sigue siendo secuencial, el speedup no puede pasar de $1 / (f + (1 - f) / p)$ aunque se agreguen todos los hilos que se quiera.

Física del screensaver. Para la gravedad usamos la ley de Newton ($a = G m / d^2$). Cuando dos cuerpos quedan casi encima uno del otro esa aceleración se vuelve enorme. Por eso sumamos una constante pequeña ϵ^2 al denominador, en las simulaciones de N cuerpos (Aarseth, 2003). Las absorciones las tratamos como choques inelásticos que conservan el momento con $v = (m_1 v_1 + m_2 v_2) / (m_1 + m_2)$.

3. Diseño e implementación

3.1 Cómo funciona la galaxia

Al inicio las N partículas forman un disco que gira alrededor del centro de la ventana. Sus colores salen de una paleta de estrellas y nebulosas. La gran mayoría es polvo pequeño. Alrededor de esas semillas empieza todo. Una partícula de 5 px de radio o más ya tiene gravedad y se come a las más pequeñas que caen dentro de ella. Cuando alcanza los 12 px pasa a ser estrella y su núcleo se va aclarando hasta quedar blanco. Al cumplirse su tiempo de vida ('—lifetime' en el código) explota. Las partículas no se pierden porque lo que fue absorbido vuelve como fragmento o reaparece como polvo. Así N se mantiene fijo y las mediciones se pueden comparar entre sí.

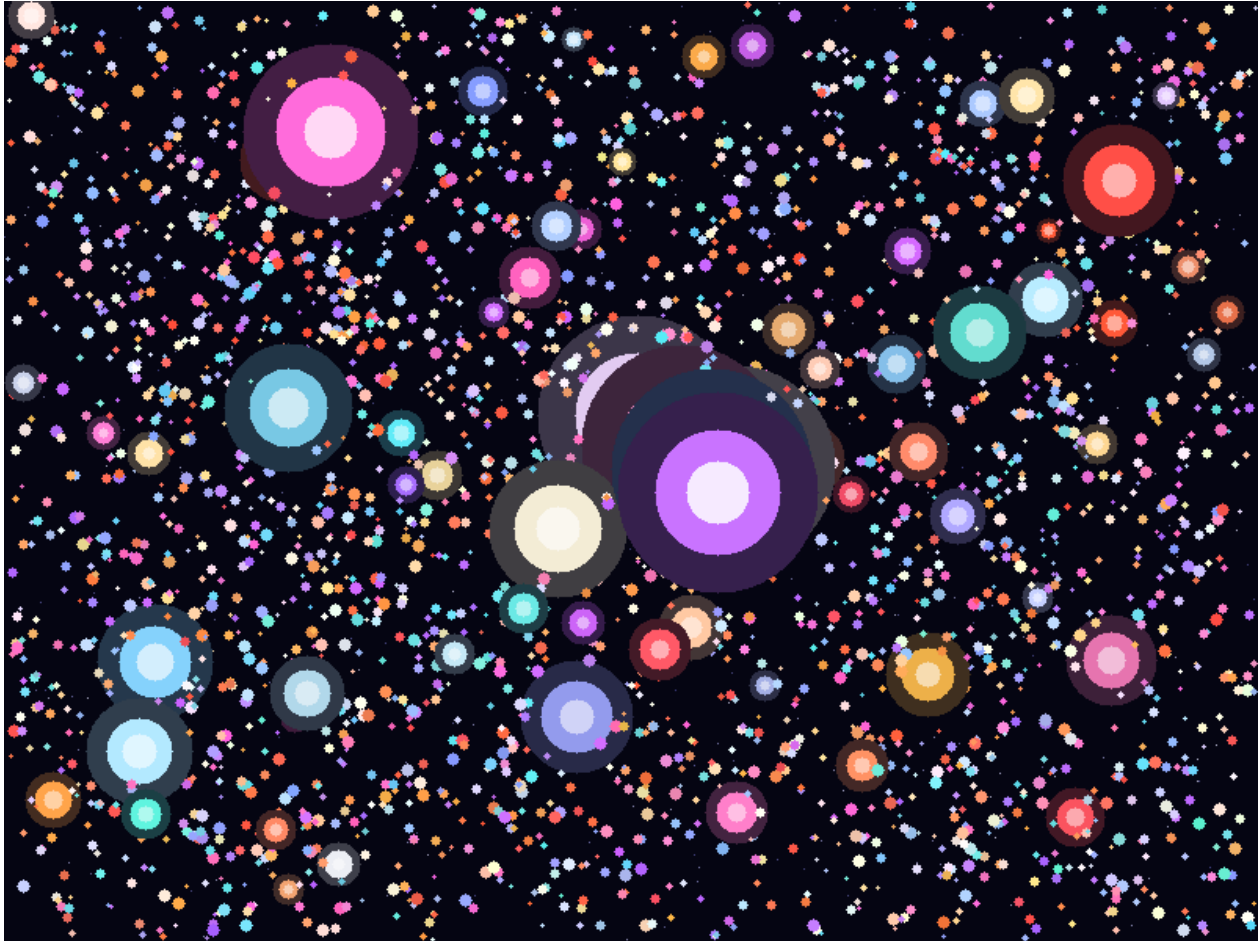


Figura 1. El screensaver en funcionamiento (versión paralela v2, $N = 5000$, 8 hilos).

3.2 Estructura del código

Decidimos que todo lo que tenía que comportarse igual en las tres versiones todo el código común estuviera en 'screensaver_common.hpp'. Ahí están los argumentos, la física, el dibujo y el ciclo principal. Las versiones ('sequential.cpp', 'parallel.cpp' y 'parallel_v2.cpp') solo cambian la forma de actualizar y de dibujar un cuadro. Si una es más rápida que otra es por cómo se asigna el trabajo. La Tabla 1 resume las fases de cada cuadro.

#	Fase	Costo	¿Paralelizable?
1	'buildAttractors': copia las estrellas con gravedad	$O(N)$	Se deja secuencial para mantener el orden
2	'updateParticle': gravedad, movimiento y rebotes	$O(N \cdot E)$	Sí, cada iteración escribe solo su partícula
3	'findPredator': quién absorbe a cada partícula	$O(N^2)$	Sí, solo lee y escribe 'predatorOf[]'
4	'resolveAbsorption': crecimiento y momento	$O(N)$	No, varias presas afectan al mismo depredador
5	'processSupernovas': explosiones y reaparición	$O(N)$	No, usa 'rand()' y toca partículas arbitrarias
6	'rasterizeRows': dibujo en memoria	$O(\text{píxeles} + N)$	Sí, por franjas de filas
7	'presentFramebuffer': mostrar en pantalla	$O(\text{píxeles})$	No, SDL solo dibuja desde el hilo principal

Tabla 1. Fases de cada cuadro. E es el número de estrellas.

La fase 7 no la podíamos repartir. Según la documentación de SDL2 las funciones de renderizado se deben llamar solo desde el hilo principal (SDL Community, 2024). Por eso decidimos dibujar primero en un arreglo de píxeles propio que sí permite ejecutar en varios hilos. Al hilo principal solo le queda enviar ese arreglo a la pantalla.

3.3 PCAM aplicado

Repartimos por partícula en las fases 2 y 3 y por filas de la imagen en la fase 6. Donde más tuvimos que pensar fue en la comunicación. Para calcular la gravedad cada partícula necesita saber dónde están todas las estrellas. Si leyera esas posiciones justo cuando otro hilo las está moviendo el resultado dependería de quién llega primero. Lo evitamos copiando antes las estrellas a un arreglo aparte llamado 'attractors' que nadie toca mientras se calcula la gravedad. Gracias a eso ningún hilo escribe en algo que otro esté leyendo y nunca necesitamos 'critical' ni 'atomic'. Las iteraciones se reparten en bloques por hilo. Las fases 4 y 5 son baratas y dependen unas partículas de otras, así que las dejamos juntas en un solo hilo. Las barreras implícitas de OpenMP se encargan de que una fase no empiece antes de que termine la anterior.

3.4 Versiones paralelas

La **v1** ('parallel.cpp') es la versión más simple posible. Tiene un '#pragma omp parallel for schedule(static) default(none)' en la fase 2 y otro en la fase 3. Elegimos 'default(none)' a propósito porque obliga a declarar qué variables se comparten. Así el compilador nos avisaba si se nos escapaba alguna. El dibujo se quedó secuencial.

Al medir la v1 descubrimos tres cosas que se podían mejorar y la **v2** ('parallel_v2.cpp') corrige las tres. La primera era que cada cuadro abría y cerraba dos regiones paralelas. Ahora hay una sola. Dentro de ella un 'omp single' hace que se congele, dos 'omp for' se encargan de las fases 2 y 3 y otro 'omp single' aplica absorciones y supernovas. Este 'single' funciona como exclusión mutua, algo necesario porque 'rand()' no debe tener seguridad entre hilos (ISO/IEC, 2018). La segunda era el reparto. Las partículas que ya fueron absorbidas terminan su búsqueda al instante y con reparto estático había hilos que terminaban mucho antes que otros. Por eso lo cambiamos a 'schedule(dynamic, 64)'. La tercera era el dibujo, que en la v2 también se distribuye. La imagen se divide en 4 franjas horizontales por hilo y cada franja la pinta un único hilo en el mismo orden que la versión secuencial.

4. Metodología de medición

Usamos siempre la misma laptop con un Intel Core i7-11800H de 8 núcleos y 16 hilos, Ubuntu 24.04, g++ 13.3 con '-O2' y SDL 2.30. Cada corrida simula 300 cuadros con el vsync apagado. Con vsync encendido el programa se sincroniza con el monitor y nosotros queríamos medir cuánto tarda el cálculo y no cuánto espera. Probamos tres tamaños ($N = 2000, 5000$ y 10000). La versión secuencial corrió con 1 hilo y las paralelas con 1, 2, 4, 8 y 16 hilos. Con cada especificación de parámetros se corrió diez veces con 'scripts/benchmark.py' y el speedup se promedia en base a esas diez corridas. En todas fijamos 'OMP_WAIT_POLICY=passive' por razones que explicamos en la discusión.

5. Resultados y discusión

5.1 Resultados

N	Secuencial	v1 – 4 hilos	v1 – 8 hilos	v1 – 16 hilos	v2 – 4 hilos	v2 – 8 hilos	v2 – 16 hilos
2000	0.699 s	0.353 s · S 1.98 · E 50 %	0.293 s · S 2.39 · E 30 %	0.260 s · S 2.69 · E 17 %	0.293 s · S 2.39 · E 60 %	0.246 s · S 2.84 · E 36 %	0.244 s · S 2.86 · E 18 %
5000	4.702 s	1.326 s · S 3.55 · E 89 %	1.022 s · S 4.60 · E 58 %	0.821 s · S 5.73 · E 36 %	1.312 s · S 3.58 · E 90 %	0.893 s · S 5.26 · E 66 %	0.704 s · S 6.68 · E 42 %
10000	19.673 s	5.867 s · S 3.35 · E 84 %	4.094 s · S 4.80 · E 60 %	3.278 s · S 6.00 · E 38 %	5.854 s · S 3.36 · E 84 %	3.624 s · S 5.43 · E 68 %	2.718 s · S 7.24 · E 45 %

Tabla 2. Tiempo promedio de 300 cuadros, speedup (S) y eficiencia (E). Cada valor es el promedio de 10 corridas.

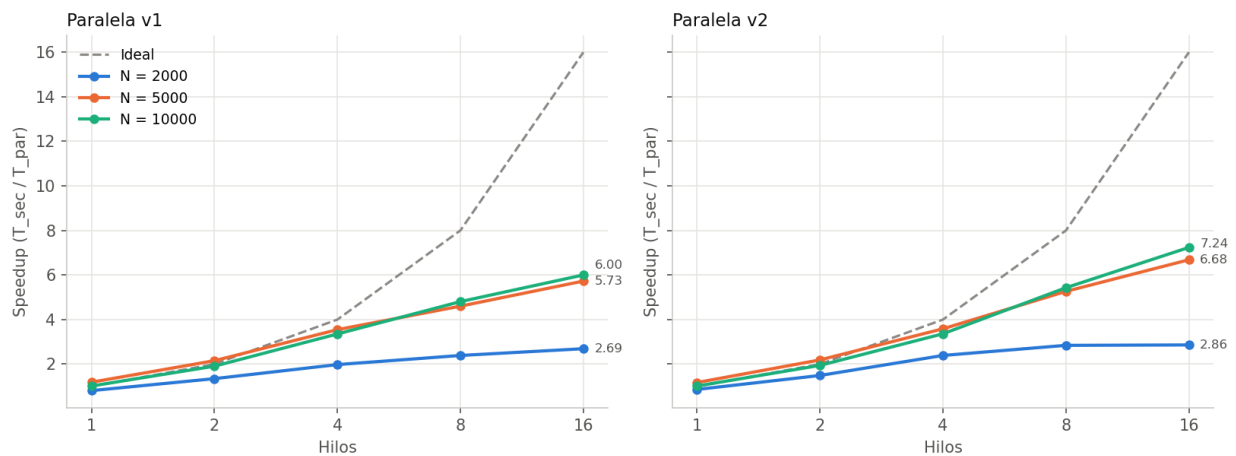


Figura 2. Speedup según el número de hilos (la línea punteada es el ideal, $S = p$).

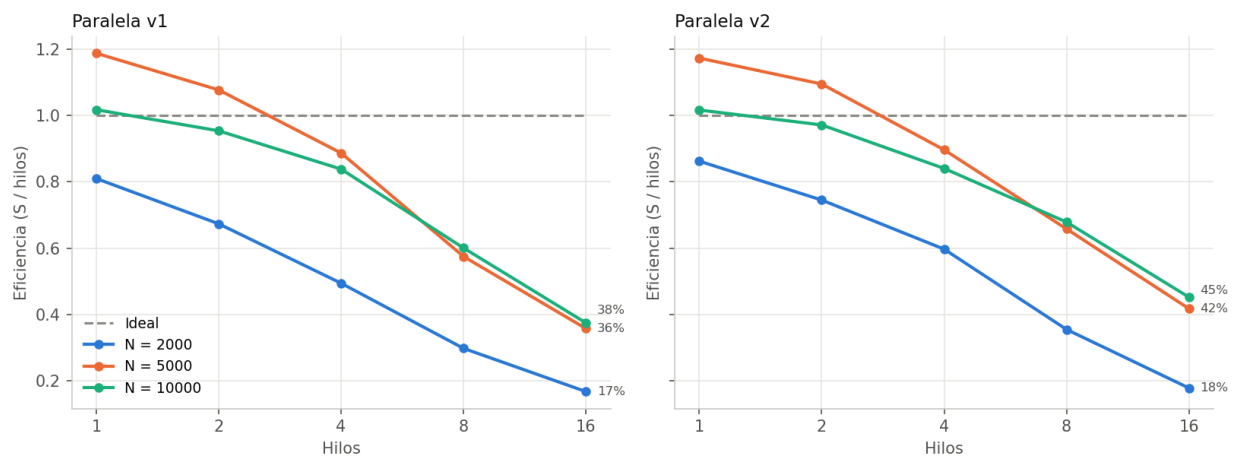


Figura 3. Eficiencia según el número de hilos.

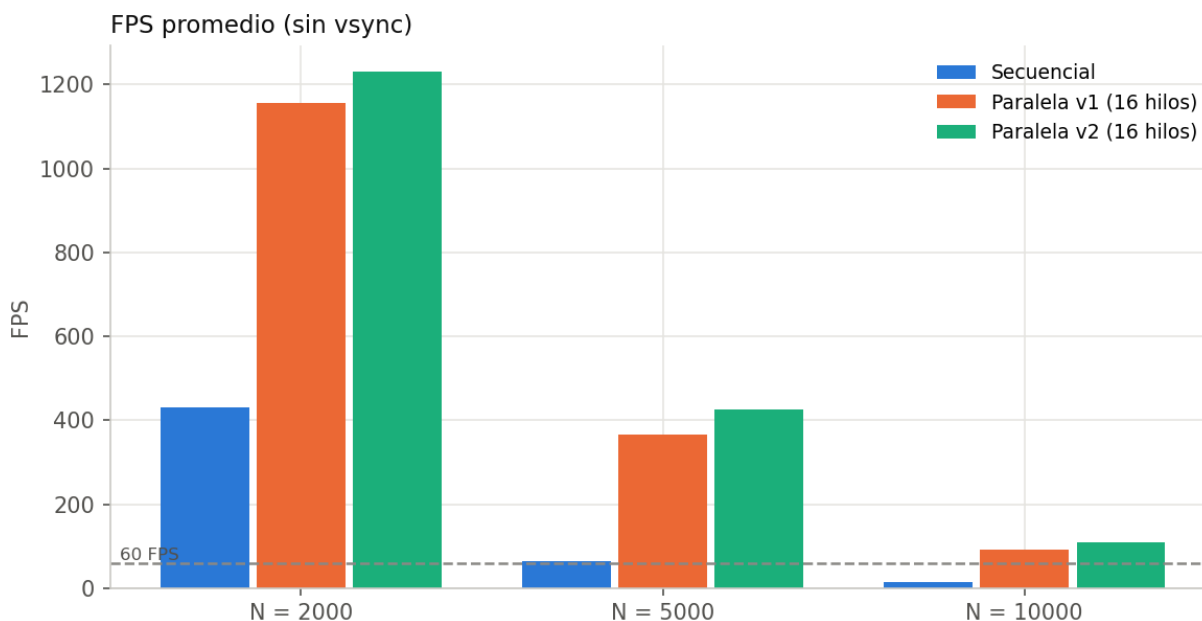


Figura 4. FPS promedio sin vsync de la versión secuencial y de las paralelas con 16 hilos.

Es necesario medir cuántos elementos se pueden dibujar sin que bajen los cuadros por segundo. Para eso el script va duplicando N mientras el programa aguante 60 FPS y después ajusta el valor con una bisección (Tabla 3).

Versión	Hilos	N máximo a ≥ 60 FPS	Relación con la secuencial
Secuencial	1	5 343	1.00×
Paralela v1	16	40 500	7.58×
Paralela v2	16	44 500	8.33×

Tabla 3. N máximo que cada versión sostiene a 60 FPS o más (5 corridas de 180 cuadros por paso).

Además comparamos las dos políticas de espera que ofrece OpenMP, con N = 5000 (Tabla 4).

Versión	Hilos	Espera activa (FPS)	Espera pasiva (FPS)
Paralela v1	8	344.7	325.2
Paralela v1	16	299.9	409.0
Paralela v2	8	425.6	409.8
Paralela v2	16	352.9	484.5

Tabla 4. FPS promedio con N = 5000 según la política de espera (10 corridas de 120 cuadros).

5.2 Discusión

Todo depende de N . Con 10000 partículas la búsqueda de depredadores crece con la complejidad algorítmica $O(N^2)$ y por lo que llena el cuadro completo. En la versión secuencial son 64.9 ms de actualización en comparación con 0.65 ms de dibujo. En ese caso los hilos sí se aprovechan y la v2 con 16 hilos llega a un speedup de 7.24. El screensaver sube de 15 a 110 FPS. Con 2000 partículas cambia el escenario. El speedup se estanca alrededor de 2.9 y agregar hilos es inútil. En cada cuadro solo hay unos 2 ms de trabajo distribuible mientras que mostrar la imagen, las fases secuenciales y la sincronización cuestan siempre lo mismo. Esto es lo que describe la ley de Amdahl. La Tabla 3 muestra lo mismo de otra forma porque a 60 FPS la v2 soporta 8.3 veces más partículas que la secuencial.

¿Valió la pena la v2? Cuando hay poco hilos no porque las dos versiones van casi iguales. A partir de 8 hilos la v2 ya es mejor (5.43 en comparación con 4.80 y 7.24 en comparación con 6.00 con $N = 10000$). Las mejoras que hicimos importan más cuando hay muchos hilos esperándose entre sí, también llama la atención cuánto baja la eficiencia al pasar de 8 a 16 hilos, esto sucede debido al procesador por que tiene 8 núcleos físicos y los otros 8 hilos son de Hyper-Threading y comparten núcleo. Esto hace que su efectividad seama apenas 1.25 veces mas.

Durante las pruebas sucedieron 3 cosas que llamaron la atención. La más importante fue que la v2 con 16 hilos y $N = 1000$ cayó de unos 1600 FPS a 40. Por defecto un hilo de OpenMP que llega a una barrera no se duerme sino que se queda girando mientras espera. Si el sistema operativo saca a cualquiera de los 16 para atender al servidor gráfico los otros 15 se quedan girando hasta que el otro hilo regrese. Con 'OMP_WAIT_POLICY=passive' el problema se fue. Según la Tabla 4 la espera pasiva da hasta 37 % más FPS con 16 hilos, aunque con 8 hilos y una espera activa sigue siendo un poco mejor, por eso medimos con la pasiva y por eso los programas usan por defecto la mitad de los procesadores. Después nos encontramos con que al principio la versión paralela con un solo hilo tardaba el doble que la secuencial, esto se daba por GCC, por que dentro de la región paralela no estaba expandiendo en línea la función 'findPredator'. Lo arreglamos con '___attribute__((always_inline))'. Por ultimo, vimos que hay mediciones de eficiencia que llegaron a tener mas de 100 % con $N = 5000$. Esto sucedia por que el compilador optimiza cada región paralela como si fuera una función aparte y con un hilo eso es suficiente para que las versiones paralelas se alejen de la version secuencial en los tamaños que son pequeños. Con $N = 10000$ la diferencia baja.

6. Conclusiones

1. Utilizar PCAM permitió paralelizar de una manera mas simple sin tener que utilizar 'critical' ni 'atomic' y calcular la gravedad de las estrellas.
2. El código de la versión 2 fue mucho mas rápido que la secuencial, aproximadamente 7 veces mas y es capaz de renderizar aproximadamente 8 veces mas partículas
3. Entre mayor sea el valor de N, se obtiene mejor speedup, por que se puede distribuir mejor como dice la Ley de Amdahl y mas cuando se llega a utilizar el Hyper-Threading.

7. Referencias

Aarseth, S. J. (2003). *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511535246>

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference* (pp. 483–485). ACM. <https://doi.org/10.1145/1465482.1465560>

Foster, I. (1995). *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Addison-Wesley. <https://www.mcs.anl.gov/~itf/dbpp/>

ISO/IEC. (2018). *ISO/IEC 9899:2018 Information technology — Programming languages — C*. International Organization for Standardization.

OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. <https://www.openmp.org/specifications/>

Pacheco, P. S. y Malensek, M. (2022). *An Introduction to Parallel Programming* (2.^a ed.). Morgan Kaufmann.

SDL Community. (2024). *Simple DirectMedia Layer 2.0 – API documentation*. <https://wiki.libsdl.org/SDL2/>

Claude – Se utilizó con motivos de mejorar redacción, organización y presentación del documento.

```
graph TD
    Inicio([Inicio]) --> Captura[/Captura de argumentos argc / argv/]
    Captura --> Argc{¿argc > 1?}
    Argc -- no --> Solicitudes[/Solicitud de datos por consola N y hilos (hasta 3 intentos)/]
    Argc -- si --> Parse[/parseArguments() strtol/strtoul, rangos, opciones conocidas, min <= max, argumentos de sobra/]
    Solicitudes --> Entrada{¿Entrada válida?}
    Parse --> Entrada
    Entrada -- no --> MensajeError[/Mensaje de error + uso (programación defensiva)/]
    MensajeError --> FinFailure([Fin (EXIT_FAILURE)])
    Entrada -- si --> Seed[/seedRandom() omp_set_num_threads(hilos)/]
    Seed --> InitSdl[/initSdl() ventana, renderer, textura/]
    InitSdl --> SdlOk{¿SDL OK?}
    SdlOk -- no --> MensajeError
    SdlOk -- si --> InitGalaxy[/initializeGalaxy() N partículas en disco giratorio, buffers y framebuffer/]
    InitGalaxy --> HandleEvents{handleEvents() ¿ESC / cerrar?}
    HandleEvents -- salir --> DestroySdl[/destroySdl() textura, renderer, ventana, SDL_Quit/]
    DestroySdl --> FinSuccess([Fin (EXIT_SUCCESS)])
    HandleEvents -- continuar --> UpdateGalaxy[/updateGalaxyParallelV2 - #pragma omp parallel (una región por frame)/]
    UpdateGalaxy --> BuildAttractors[/omp single: buildAttractors() copia inmutable de estrellas/]
    BuildAttractors --> Barrier1[barrera implícita]
    Barrier1 --> UpdateParticle[/omp for static: updateParticle(i) gravedad, movimiento, rebotes, edad/]
    UpdateParticle --> Barrier2[barrera implícita]
    Barrier2 --> FindPredator[/omp for dynamic,64: predatorOf[i] = findPredator(i) O(N²), solo lectura/]
    FindPredator --> Barrier3[barrera implícita]
    Barrier3 --> ResolveMutualExclusion[/omp single (exclusión mutua): resolveAbsorption() + processSupernovas() (usa rand(), no thread-safe)/]
    ResolveMutualExclusion --> Barrier4[barrera implícita + join]
    Barrier4 --> RenderGalaxy[/renderGalaxyParallelV2 - #pragma omp parallel for dynamic 4 franjas por hilo; cada franja la pinta un solo hilo/]
    RenderGalaxy --> Barrier5[barrera implícita + join]
    Barrier5 --> PresentFrame[/presentFramebuffer() hilo principal (SDL no es thread-safe)/]
    PresentFrame --> FPS[/Despliegue de FPS titulo de ventana + consola/]
    FPS --> Benchmark{¿Modo benchmark y frames completos?}
    Benchmark -- si --> Results[/Despliegue de resultados tiempo, FPS, ms/fase, checksum, fila CSV/]
    Results --> DestroySdl
    Benchmark -- no --> HandleEvents
```

Figura A1. Diagrama de flujo del screensaver (versión paralela v2).

Anexo 2 – Catálogo de funciones

A2.1 Funciones compartidas ('savesaver_common.hpp')

Función	Entradas	Salidas	Descripción
'parseArguments'	'argc: int', 'argv: char*[]' (argumentos) 'supportsThreads: bool' (acepta hilos) 'defaultThreads: int'	'config: Config&' (parámetros validados) retorno 'bool' (true si son válidos)	Recorre los argumentos posicionales ('N', 'hilos') y las opciones ('--width', '--gravity', '--frames', ...). Valida cada valor con 'parseLongInRange'/'parseFloatInRange' y las relaciones con 'validateConfig'. Sin argumentos, solicita N e hilos con 'promptLong'.
'parseLongInRange' / 'parseFloatInRange'	'text: const char*' 'minValue', 'maxValue: long/float' 'name: const char*' (para el mensaje)	'outValue: long&/float&' retorno 'bool'	Convierte con 'strtol'/'strtod' y rechaza cadena vacía, basura al final, desbordamiento ('ERANGE'), valores no finitos y fuera de rango. Imprime el error en 'stderr'.
'promptLong'	'question: const char*' rango 'name'	'outValue: long&' retorno 'bool'	Solicita un entero por consola con 'fgets' y lo valida. Hasta 3 intentos.
'validateConfig'	'config: const Config&'	retorno 'bool'	Verifica relaciones entre parámetros (mínimos $\leq$ máximos, '--csv' sin '--frames').
'printUsage'	'programName: const char*' 'supportsThreads: bool'	texto en 'stderr'	Muestra la ayuda de uso con rangos y valores por defecto.
'seedRandom'	'config: Config&'	'config.seed' estado de 'rand()'	Inicializa 'srand' con la semilla dada o con la hora actual.
'randomRange'	'minValue', 'maxValue: float'	retorno 'float'	Número pseudoaleatorio uniforme en el intervalo (usa 'rand()', solo desde un hilo).
'randomGalaxyColor'	—	retorno 'UInt32' (ARGB)	Color de la paleta galáctica con variación aleatoria de ± 25 por canal.
'packColor' / 'lerpColor'	canales 'r, g, b: UInt8' / 'from, to: UInt32', 't: float'	retorno 'UInt32'	Empaqueta un color ARGB8888 / interpola linealmente entre dos colores.
'randomBirthRadius'	'config: const Config&'	retorno 'float'	Radio de nacimiento: polvo en [min, max] o, con probabilidad 3 %, protoestrella (5–8 px).
'makeGalaxyDust'	'config: const Config&'	retorno 'Particle'	Crea una partícula en un disco centrado, con velocidad tangencial (ángulo + $\pi/2$) para el giro galáctico.
'respawnAsDust'	'p: Particle&' 'config'	'p' modificada	Reutiliza una partícula absorbida como polvo en una posición y dirección aleatorias.

Función	Entradas	Salidas	Descripción
'resetLife' / 'clampInsideCanvas'	'p: Particle&' 'config' o 'width, height'	'p' modificada	Reinicia edad y tiempo de vida ($\pm 30\%$) / mantiene el centro dentro del canvas.
'makeBackgroundStars'	'width, height: int'	retorno 'vector<BackgroundStar>'	Fondo de estrellas fijas generado con un LCG propio (no altera la secuencia de 'rand()').
'initializeGalaxy'	'config: const Config&'	'galaxy: Galaxy&'	Crea las N partículas y reserva 'attractors', 'predatorOf', 'freeSlots' y el <i>framebuffer</i> (una sola vez).
'buildAttractors'	'galaxy: Galaxy&'	'galaxy.attractors'	Copia (x, y, masa, índice) de cada partícula activa con radio ≥ 5 a un arreglo inmutable durante la fase de gravedad.
'updateParticle'	'p: Particle&' 'index: int' 'attractors: const vector<Attractor>&' 'deltaTimeSeconds : float' 'config'	'p' actualizada	<i>Kernel</i> paralelizable. Suma $G m (dx, dy) / (d^2 + \epsilon^2)^{3/2}$ de cada estrella, integra velocidad y posición, limita la rapidez a 450 px/s, rebota en los bordes y aumenta la edad si es estrella.
'findPredator'	'particles: const vector<Particle>&' 'j: int'	retorno 'int' (índice del depredador o -1)	<i>Kernel</i> paralelizable $O(N)$. Busca la partícula activa de mayor radio (≥ 5 y $\geq r_j + 0.5$) cuyo disco contiene el centro de j. Solo lectura.
'resolveAbsorption'	'galaxy: Galaxy&' (usa 'predatorOf') 'config'	partículas modificadas	Secuencial. Aplica cada absorción: momento conservado, área + 80 % del área de la presa (radio máx. 40) y la presa queda inactiva.
'explodeStar'	'galaxy: Galaxy&' 'starIndex: int' 'config'	partículas modificadas	Reparte el 90 % del área de la estrella en hasta 40 fragmentos, tomados de 'freeSlots', con velocidad radial en ángulos $2\pi k/K$.
'processSupernovas'	'galaxy: Galaxy&' 'config'	partículas modificadas	Secuencial. Lista las partículas libres, explota las estrellas que cumplieron su vida y hace reaparecer el polvo absorbido hace > 1.5 s.
'computeChecksum'	'particles: const vector<Particle>&'	retorno 'double'	Suma de verificación del estado ($\sum x + 3y + 7r$) para comprobar que las tres versiones coinciden.
'fillDiscRows'	'pixels: vector<UInt32>&' 'width' centro, radio: 'float' 'color' 'rowBegin, rowEnd: int'	píxeles modificados	Rellena un disco por barrido de líneas horizontales (semiancho $\sqrt{(r^2 - dy^2)}$), limitado a las filas indicadas.

Función	Entradas	Salidas	Descripción
'rasterizeRows'	'galaxy: Galaxy& 'config' 'rowBegin, rowEnd: int'	'galaxy.framebuffer' (filas indicadas)	Limpia las filas, dibuja el fondo de estrellas y las partículas activas en orden. Las estrellas llevan halo y un núcleo que se blanquea con la edad. Llamadas con filas disjuntas pueden ejecutarse en paralelo.
'presentFramebuffer'	'sdl: SdlContext& 'pixels' 'width'	cuadro en pantalla	'SDL_UpdateTexture' + 'SDL_RenderCopy' + 'SDL_RenderPresent' (solo hilo principal).
'initSdl' / 'destroySdl'	'config' 'title: const char*' / 'sdl: SdlContext&'	'sdl' (ventana, renderer, textura) retorno 'bool'	Crea los recursos de SDL (con renderer por software de respaldo) / los libera en orden inverso, tolerando punteros nulos.
'handleEvents'	—	retorno 'bool' (false = salir)	Procesa la cola de eventos: cierre de ventana o tecla ESC.
'FpsCounter::tick'	'frameSeconds: double' 'window' 'versionName' 'config' 'showThreads', 'printToConsole: bool'	título de ventana, consola 'lastFps'	Cada 0.5 s calcula FPS = cuadros / tiempo y los despliega.
'reportBenchmark'	'versionName' 'config' 'threads' 'totalSeconds: double' 'timings: FrameTimings' 'checksum'	texto en 'stdout' fila CSV	Imprime tiempo, FPS promedio, ms por cuadro de cada fase y checksum. Agrega la fila al CSV si se usó '--csv'.
'runScreensaver<UpdateFn, RenderFn>'	'config' 'versionName' 'showThreads' 'updateStep(galaxy, dt)' 'renderStep(galaxy)'	retorno 'int' (código de salida)	Ciclo principal común: SDL → galaxia → [eventos → update → render → present → FPS] hasta ESC o hasta completar '--frames'. Reporta y libera recursos.

Tabla A1. Funciones compartidas por las tres versiones.

A2.2 Funciones propias de cada versión

Función (archivo)	Entradas	Salidas	Descripción
'updateGalaxySequential' ('sequential.cpp')	'galaxy: Galaxy&' 'deltaTimeSeconds: float' 'config'	'galaxy' actualizada	Ejecuta las fases 1–5 en un solo hilo.
'renderGalaxySequential' ('sequential.cpp')	'galaxy' 'config'	'framebuffer'	'rasterizeRows(0, height)'.
'updateGalaxyParallelV1' ('parallel.cpp')	'galaxy' 'deltaTimeSeconds' 'config' 'threadCount: int'	'galaxy' actualizada	Fases 2 y 3 con '#pragma omp parallel for schedule(static) default(none)'. Fases 1, 4 y 5 secuenciales.
'renderGalaxyParallelV1' ('parallel.cpp')	'galaxy' 'config'	'framebuffer'	Rasterizado secuencial.
'updateGalaxyParallelV2' ('parallel_v2.cpp')	'galaxy' 'deltaTimeSeconds' 'config' 'threadCount'	'galaxy' actualizada	Una región 'omp parallel': 'single' (fase 1) → 'for static' (fase 2) → 'for dynamic,64' (fase 3) → 'single' (fases 4–5), sincronizadas por barreras implícitas.
'renderGalaxyParallelV2' ('parallel_v2.cpp')	'galaxy' 'config' 'threadCount'	'framebuffer'	'parallel for schedule(dynamic,1)' sobre $4 \cdot p$ franjas horizontales (mínimo 4 filas cada una).
'defaultThreadCount' ('parallel*.cpp')	—	retorno 'int'	Hilos por defecto: mitad de los procesadores lógicos ('omp_get_num_procs()/2').
'warnIfOversubscribed' ('parallel*.cpp')	'threadCount: int'	aviso en 'stderr'	Advierte si hilos $\geq$ procesadores lógicos sin 'OMP_WAIT_POLICY' definida.
'main' (los tres archivos)	'argc', 'argv'	código de salida	Valida argumentos, fija la semilla y los hilos ('omp_set_num_threads') y llama a 'runSaver' con las funciones de su versión.

Tabla A2. Funciones propias de cada versión.

A2.3 Estructuras de datos principales

Estructura	Campos	Uso
'Config'	'particleCount', 'threadCount', 'width', 'height', 'minRadius', 'maxRadius', 'minSpeed', 'maxSpeed', 'gravity', 'lifetime', 'benchFrames', 'seed', 'vsync', 'csvPath'	Parámetros de ejecución leídos de la línea de comandos.
'Particle'	'x, y, vx, vy, radius, age, lifetime, deadTime: float' 'color: Uint32' 'active: bool'	Estado de cada partícula. Masa = 'radius ² '.
'Attractor'	'x, y, mass: float' 'index: int'	Copia inmutable de una estrella para la fase de gravedad.
'Galaxy'	'particles', 'attractors', 'predatorOf', 'freeSlots', 'framebuffer', 'backgroundStars'	Todo el estado de la simulación y los <i>buffers</i> reutilizados cada cuadro.
'SdlContext'	'window', 'renderer', 'texture'	Recursos de SDL.
'FrameTimings'	'updateSeconds', 'renderSeconds', 'frames'	Acumuladores para el reporte del benchmark.

Tabla A3. Estructuras de datos principales.

Anexo 3 – Bitácora de pruebas

Las pruebas se corrieron con 'python3 scripts/benchmark.py' como se describe en la sección 4. Quien quiera revisar los datos crudos los encuentra en el repositorio. Cada corrida está en 'results/mediciones.csv' y lo que imprimió la consola está en 'results/salida_consola.txt'.

A3.1 Mediciones individuales (10 por prueba)

Aquí están las 10 mediciones de cada prueba en segundos para 300 cuadros. Para que quepan en la página la tabla va en dos partes (t1 a t5 y t6 a t10). Sec. es la versión secuencial y v1 y v2 las paralelas. El promedio, el speedup y la eficiencia de cada prueba están en la Tabla A5.

Versión	N	Hilos	t1	t2	t3	t4	t5
Sec.	2000	1	0.676	0.680	0.728	0.739	0.695
v1	2000	1	0.841	0.912	0.866	0.905	0.846
v1	2000	2	0.542	0.541	0.489	0.574	0.498
v1	2000	4	0.365	0.359	0.375	0.349	0.345
v1	2000	8	0.298	0.293	0.289	0.291	0.286
v1	2000	16	0.255	0.259	0.247	0.274	0.258
v2	2000	1	0.795	0.780	0.776	0.780	0.863
v2	2000	2	0.551	0.446	0.447	0.480	0.446
v2	2000	4	0.297	0.293	0.295	0.294	0.289
v2	2000	8	0.238	0.247	0.248	0.233	0.247
v2	2000	16	0.244	0.249	0.246	0.247	0.257
Sec.	5000	1	4.957	4.953	4.798	4.954	4.476
v1	5000	1	4.140	3.991	4.089	3.960	3.946
v1	5000	2	2.154	2.275	2.105	2.108	2.115
v1	5000	4	1.329	1.222	1.368	1.350	1.366
v1	5000	8	0.986	1.034	1.085	1.024	1.004
v1	5000	16	0.791	0.804	0.891	0.801	0.792
v2	5000	1	3.839	4.008	3.860	4.269	3.984
v2	5000	2	2.041	2.166	2.214	2.237	2.105
v2	5000	4	1.249	1.385	1.353	1.246	1.290
v2	5000	8	0.854	0.870	0.918	0.876	0.950
v2	5000	16	0.718	0.687	0.735	0.694	0.696
Sec.	10000	1	19.601	20.134	20.399	19.132	19.198
v1	10000	1	19.274	19.648	19.154	19.599	19.186
v1	10000	2	10.496	9.960	10.118	10.708	10.876

Versión	N	Hilos	t1	t2	t3	t4	t5
v1	10000	4	5.468	6.243	5.685	5.868	5.929
v1	10000	8	4.045	3.941	4.125	3.975	4.027
v1	10000	16	3.330	3.236	3.350	3.256	3.274
v2	10000	1	20.949	18.882	18.866	20.213	18.978
v2	10000	2	10.159	10.270	10.435	10.217	9.798
v2	10000	4	5.894	5.816	5.989	5.738	5.896
v2	10000	8	3.549	3.568	3.667	3.624	3.695
v2	10000	16	2.736	2.671	2.782	2.686	2.758
Versión	N	Hilos	t6	t7	t8	t9	t10
Sec.	2000	1	0.696	0.689	0.677	0.730	0.676
v1	2000	1	0.835	0.881	0.828	0.829	0.884
v1	2000	2	0.517	0.509	0.511	0.505	0.500
v1	2000	4	0.357	0.352	0.354	0.341	0.334
v1	2000	8	0.289	0.293	0.298	0.295	0.294
v1	2000	16	0.260	0.256	0.254	0.256	0.275
v2	2000	1	0.822	0.822	0.838	0.783	0.843
v2	2000	2	0.463	0.450	0.451	0.483	0.471
v2	2000	4	0.299	0.289	0.283	0.285	0.301
v2	2000	8	0.247	0.241	0.255	0.244	0.259
v2	2000	16	0.244	0.241	0.263	0.234	0.218
Sec.	5000	1	4.526	4.482	4.877	4.501	4.493
v1	5000	1	3.972	3.827	3.881	3.829	3.968
v1	5000	2	2.171	2.322	2.184	2.213	2.186
v1	5000	4	1.303	1.280	1.308	1.371	1.359
v1	5000	8	0.999	0.944	1.047	1.039	1.057
v1	5000	16	0.800	0.796	0.894	0.817	0.826
v2	5000	1	4.143	3.829	3.976	3.974	4.218
v2	5000	2	2.229	2.093	2.157	2.057	2.182
v2	5000	4	1.302	1.313	1.356	1.322	1.300
v2	5000	8	0.890	0.887	0.920	0.888	0.880
v2	5000	16	0.698	0.699	0.698	0.702	0.708
Sec.	10000	1	19.319	19.526	20.157	19.227	20.036
v1	10000	1	19.082	19.217	19.684	19.278	19.294
v1	10000	2	10.312	10.060	10.030	10.005	10.591
v1	10000	4	5.616	6.032	5.619	5.997	6.217
v1	10000	8	4.200	4.146	4.253	4.220	4.011
v1	10000	16	3.269	3.321	3.227	3.263	3.253
v2	10000	1	19.010	19.872	18.950	18.937	18.952

Versión	N	Hilos	t1	t2	t3	t4	t5
v2	10000	2	9.888	9.918	10.563	9.845	10.183
v2	10000	4	6.047	5.915	5.790	5.629	5.828
v2	10000	8	3.651	3.664	3.545	3.671	3.601
v2	10000	16	2.674	2.718	2.727	2.733	2.698

Tabla A4. Tiempo de cada una de las 10 corridas por prueba (segundos).

A3.2 Resumen por prueba

Equipo: 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz - 16 hilos lógicos - Linux 6.8.0-45-generic

Cada fila = promedio de 10 corridas de 300 frames (dt fijo 1/60 s, sin vsync, semilla 2026, OMP_WAIT_POLICY=passive).

Checksums idénticos entre versiones: **sí**

Versión	N	Hilos	Tiempo prom. (s)	Desv. (s)	FPS prom.	Update (ms)	Render (ms)	Speedup	Eficiencia
Secuencial	2000	1	0.6986	0.0244	429.9	2.020	0.303	1.00	100.0%
Paralela v1	2000	1	0.8626	0.0315	348.2	2.567	0.303	0.81	81.0%
Paralela v1	2000	2	0.5186	0.0261	579.8	1.412	0.310	1.35	67.4%
Paralela v1	2000	4	0.3531	0.0120	850.5	0.842	0.329	1.98	49.5%
Paralela v1	2000	8	0.2926	0.0039	1025.3	0.643	0.327	2.39	29.8%
Paralela v1	2000	16	0.2596	0.0086	1156.8	0.506	0.354	2.69	16.8%
Paralela v2	2000	1	0.8103	0.0314	370.7	2.339	0.356	0.86	86.2%
Paralela v2	2000	2	0.4689	0.0323	642.3	1.259	0.298	1.49	74.5%
Paralela v2	2000	4	0.2925	0.0060	1026.0	0.695	0.274	2.39	59.7%
Paralela v2	2000	8	0.2458	0.0075	1221.4	0.520	0.293	2.84	35.5%
Paralela v2	2000	16	0.2442	0.0122	1231.5	0.465	0.341	2.86	17.9%
Secuencial	5000	1	4.7019	0.2227	63.9	15.223	0.443	1.00	100.0%
Paralela v1	5000	1	3.9606	0.1007	75.8	12.778	0.416	1.19	118.7%
Paralela v1	5000	2	2.1835	0.0713	137.5	6.850	0.422	2.15	107.7%
Paralela v1	5000	4	1.3257	0.0481	226.6	3.961	0.453	3.55	88.7%
Paralela v1	5000	8	1.0220	0.0400	294.0	2.924	0.476	4.60	57.5%
Paralela v1	5000	16	0.8210	0.0390	366.1	2.217	0.512	5.73	35.8%
Paralela v2	5000	1	4.0101	0.1547	74.9	12.832	0.527	1.17	117.3%
Paralela v2	5000	2	2.1481	0.0706	139.8	6.742	0.411	2.19	109.4%
Paralela v2	5000	4	1.3116	0.0448	229.0	3.956	0.408	3.58	89.6%
Paralela v2	5000	8	0.8933	0.0283	336.1	2.560	0.410	5.26	65.8%
Paralela v2	5000	16	0.7035	0.0138	426.6	1.899	0.437	6.68	41.8%

Versión	N	Hilos	Tiempo prom. (s)	Desv. (s)	FPS prom.	Update (ms)	Render (ms)	Speedup	Eficiencia
Secuencial	10000	1	19.6730	0.4684	15.3	64.911	0.650	1.00	100.0%
Paralela v1	10000	1	19.3414	0.2186	15.5	63.823	0.636	1.02	101.7%
Paralela v1	10000	2	10.3155	0.3308	29.1	33.758	0.617	1.91	95.4%
Paralela v1	10000	4	5.8674	0.2642	51.2	18.896	0.652	3.35	83.8%
Paralela v1	10000	8	4.0944	0.1093	73.3	12.936	0.689	4.80	60.1%
Paralela v1	10000	16	3.2779	0.0416	91.5	10.210	0.706	6.00	37.5%
Paralela v2	10000	1	19.3610	0.7280	15.5	63.745	0.780	1.02	101.6%
Paralela v2	10000	2	10.1275	0.2595	29.6	33.141	0.607	1.94	97.1%
Paralela v2	10000	4	5.8540	0.1215	51.3	18.924	0.580	3.36	84.0%
Paralela v2	10000	8	3.6236	0.0547	82.8	11.486	0.583	5.43	67.9%
Paralela v2	10000	16	2.7182	0.0364	110.4	8.458	0.593	7.24	45.2%

Tabla A5. Resumen por prueba con promedio, desviación, FPS, tiempos por fase, speedup y eficiencia.

A3.3 Capturas de las mediciones

```

Bitacora-benchmark
$ ./bin/screensaver_sequential 5000 --seed 2026 --frames 300
Galaxia Secuencial: N=5000 (1 hilo), canvas 800x600, semilla=2026, modo benchmark
[Secuencial] N=5000 hilos=1 frames=300 tiempo=4.4852 s FPS=66.89 update=14.513 ms/frame render=0.427 ms/frame checksum=2835466.1556
$ ./bin/screensaver_parallel_v2 5000 1 --seed 2026 --frames 300
Galaxia Paralela v2: N=5000, canvas 800x600, semilla=2026, modo benchmark
Hilos OpenMP: 1
[Paralela v2] N=5000 hilos=1 frames=300 tiempo=4.2696 s FPS=70.26 update=13.644 ms/frame render=0.572 ms/frame checksum=2835466.1556
$ ./bin/screensaver_parallel_v2 5000 8 --seed 2026 --frames 300
Galaxia Paralela v2: N=5000, canvas 800x600, semilla=2026, modo benchmark
Hilos OpenMP: 8
[Paralela v2] N=5000 hilos=8 frames=300 tiempo=0.8634 s FPS=347.47 update=2.463 ms/frame render=0.403 ms/frame checksum=2835466.1556
$ ./bin/screensaver_parallel_v2 5000 16 --seed 2026 --frames 300
Galaxia Paralela v2: N=5000, canvas 800x600, semilla=2026, modo benchmark
Hilos OpenMP: 16
[Paralela v2] N=5000 hilos=16 frames=300 tiempo=0.7046 s FPS=425.80 update=1.872 ms/frame render=0.470 ms/frame checksum=2835466.1556
LISTO

```

Figura A3.1. Captura de la consola al ejecutar el modo benchmark con N = 5000 y 300 cuadros (versión secuencial y paralela v2 con 1, 8 y 16 hilos y 'OMP_WAIT_POLICY=passive'). El checksum es idéntico en todas las corridas.

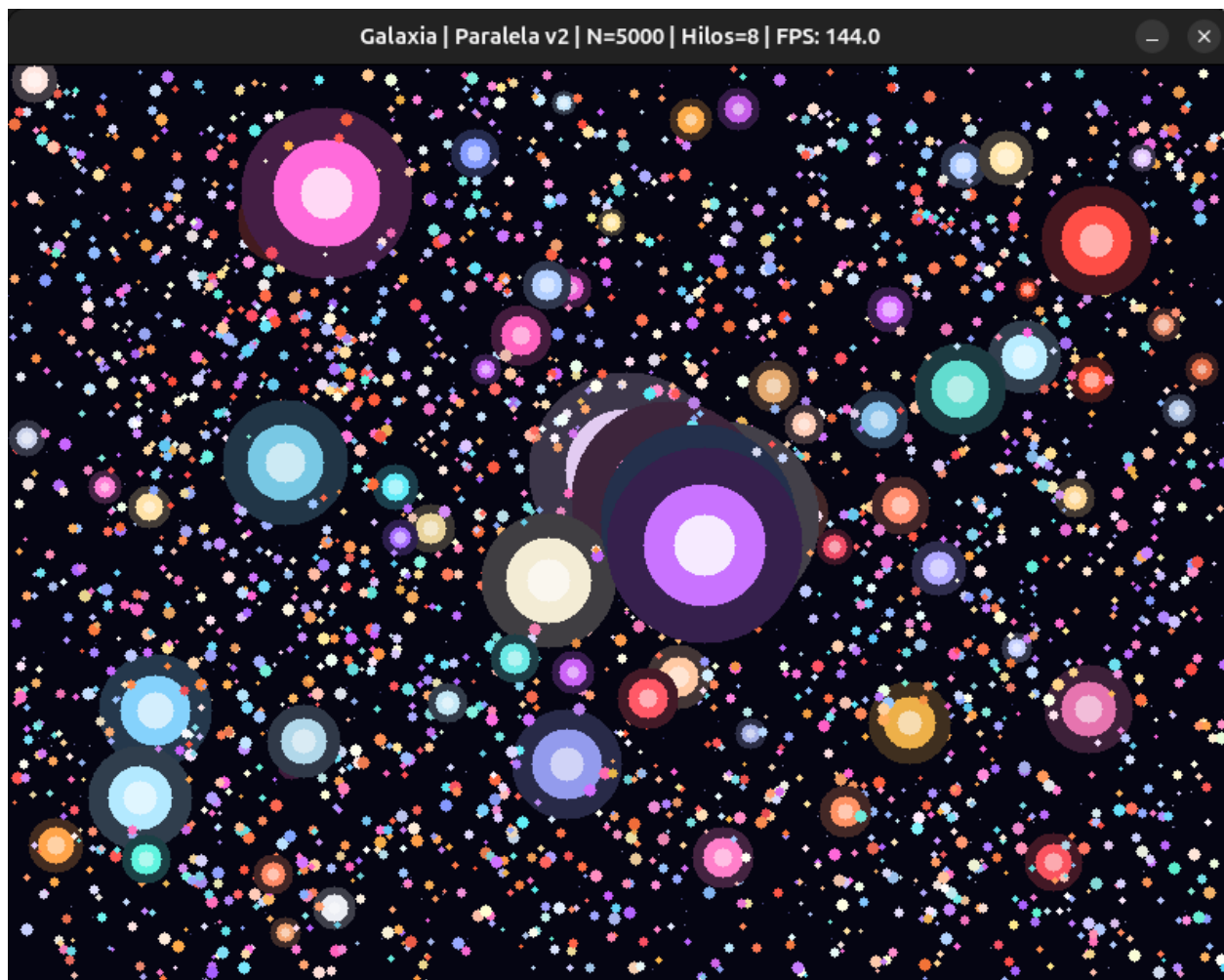


Figura A3.2. Ventana del screensaver en modo interactivo (paralela v2, N = 5000, 8 hilos) con el FPS en la barra de título (con vsync activo el FPS queda limitado a la frecuencia del monitor, 144 Hz).

A3.4 Búsqueda del N máximo a 60 FPS

N máximo con FPS promedio ≥ 60 (promedio de 5 corridas de 180 frames, sin vsync, semilla 2026, OMP_WAIT_POLICY=passive).

Versión	Hilos	N máximo a ≥ 60 FPS	Relación vs. secuencial
Secuencial	1	5343	1.00×
Paralela v1	16	40500	7.58×
Paralela v2	16	44500	8.33×

Tabla A6. N máximo a 60 FPS por versión.

Secuencial (1 hilos) N=1000: 1158.7 FPS -> OK

Secuencial (1 hilos) N=2000: 432.9 FPS -> OK

Secuencial (1 hilos) N=4000: 133.1 FPS -> OK

Secuencial (1 hilos) N=8000: 21.6 FPS -> bajo

Secuencial (1 hilos) N=6000: 43.5 FPS -> bajo
 Secuencial (1 hilos) N=5000: 74.3 FPS -> OK
 Secuencial (1 hilos) N=5500: 55.7 FPS -> bajo
 Secuencial (1 hilos) N=5250: 65.0 FPS -> OK
 Secuencial (1 hilos) N=5375: 59.6 FPS -> bajo
 Secuencial (1 hilos) N=5312: 61.2 FPS -> OK
 Secuencial (1 hilos) N=5343: 60.9 FPS -> OK
 Paralela v1 (16 hilos) N=1000: 1606.4 FPS -> OK
 Paralela v1 (16 hilos) N=2000: 1148.2 FPS -> OK
 Paralela v1 (16 hilos) N=4000: 560.7 FPS -> OK
 Paralela v1 (16 hilos) N=8000: 136.7 FPS -> OK
 Paralela v1 (16 hilos) N=16000: 70.6 FPS -> OK
 Paralela v1 (16 hilos) N=32000: 74.3 FPS -> OK
 Paralela v1 (16 hilos) N=64000: 34.7 FPS -> bajo
 Paralela v1 (16 hilos) N=48000: 50.3 FPS -> bajo
 Paralela v1 (16 hilos) N=40000: 62.2 FPS -> OK
 Paralela v1 (16 hilos) N=44000: 54.9 FPS -> bajo
 Paralela v1 (16 hilos) N=42000: 57.4 FPS -> bajo
 Paralela v1 (16 hilos) N=41000: 59.4 FPS -> bajo
 Paralela v1 (16 hilos) N=40500: 60.6 FPS -> OK
 Paralela v1 (16 hilos) N=40750: 59.7 FPS -> bajo
 Paralela v2 (16 hilos) N=1000: 1776.6 FPS -> OK
 Paralela v2 (16 hilos) N=2000: 1207.1 FPS -> OK
 Paralela v2 (16 hilos) N=4000: 603.4 FPS -> OK
 Paralela v2 (16 hilos) N=8000: 154.9 FPS -> OK
 Paralela v2 (16 hilos) N=16000: 79.9 FPS -> OK
 Paralela v2 (16 hilos) N=32000: 84.9 FPS -> OK
 Paralela v2 (16 hilos) N=64000: 37.2 FPS -> bajo
 Paralela v2 (16 hilos) N=48000: 54.7 FPS -> bajo
 Paralela v2 (16 hilos) N=40000: 69.2 FPS -> OK
 Paralela v2 (16 hilos) N=44000: 61.4 FPS -> OK
 Paralela v2 (16 hilos) N=46000: 57.9 FPS -> bajo
 Paralela v2 (16 hilos) N=45000: 59.6 FPS -> bajo
 Paralela v2 (16 hilos) N=44500: 60.3 FPS -> OK
 Paralela v2 (16 hilos) N=44750: 59.7 FPS -> bajo

Figura A3.3. Pasos de la búsqueda del N máximo (FPS promedio de 5 corridas por paso).

A3.5 Experimento de política de espera

Espera activa ('--wait-policy active')

Equipo: 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz - 16 hilos lógicos - Linux 6.8.0-45-generic

Cada fila = promedio de 10 corridas de 120 frames (dt fijo 1/60 s, sin vsync, semilla 2026, OMP_WAIT_POLICY=active).

Checksums idénticos entre versiones: **sí**

Versión	N	Hilos	Tiempo prom. (s)	Desv. (s)	FPS prom.	Update (ms)	Render (ms)	Speedup	Eficiencia
Secuencial	5000	1	1.4096	0.0755	85.4	11.286	0.451	1.00	100.0%
Paralela v1	5000	8	0.3501	0.0270	344.7	2.377	0.531	4.03	50.3%
Paralela v1	5000	16	0.4019	0.0277	299.9	2.654	0.680	3.51	21.9%
Paralela v2	5000	8	0.2822	0.0092	425.6	1.954	0.388	4.99	62.4%
Paralela v2	5000	16	0.3433	0.0378	352.9	2.250	0.597	4.11	25.7%

Tabla A7. Resultados con espera activa (N = 5000).

Espera pasiva ('--wait-policy passive')

Equipo: 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz - 16 hilos lógicos - Linux 6.8.0-45-generic

Cada fila = promedio de 10 corridas de 120 frames (dt fijo 1/60 s, sin vsync, semilla 2026, OMP_WAIT_POLICY=passive).

Checksums idénticos entre versiones: **sí**

Versión	N	Hilos	Tiempo prom. (s)	Desv. (s)	FPS prom.	Update (ms)	Render (ms)	Speedup	Eficiencia
Secuencial	5000	1	1.3403	0.0608	89.7	10.732	0.427	1.00	100.0%
Paralela v1	5000	8	0.3699	0.0193	325.2	2.619	0.455	3.62	45.3%
Paralela v1	5000	16	0.2936	0.0083	409.0	1.929	0.508	4.57	28.5%
Paralela v2	5000	8	0.2934	0.0142	409.8	2.049	0.387	4.57	57.1%
Paralela v2	5000	16	0.2478	0.0060	484.5	1.631	0.424	5.41	33.8%

Tabla A8. Resultados con espera pasiva (N = 5000).